

AUIPivotReact.jsx 서브 컴포넌트 설명서

AUIPivotReact.jsx 는 React.js 라이브러리에서 AUIGrid 를 쉽게 사용할 수 있도록 작성한 컴포넌트 입니다.

1. 정의 가능한 속성들

속성명	유형(Type)	기본값	설명
autoResize	Boolean	true	window 리사이징 시 부모에 맞게 리사이징 할지 여부를 지정합니다.
createOnMounted	Boolean	true	Mount 라이프사이클 단계에서 자동으로 피벗을 생성시킬지 여부를 지정합니다.
pivotProps	Object	{}	피벗 그리드 속성을 지정합니다.
name	String	''	피벗 그리드 이름을 지정합니다.
resizeDelayTime	Number	300	autoResize 설정 시 해당 시간 이후 리사이징을 실행합니다.

* name 속성은 의무사항이 아닙니다. 그러나 name 을 지정하면 해당 name 값을 id 로 갖는 그리드가 생성됩니다. 예로 name='showcase01' 로 설정했다면 해당 그리드는 'aui-pivot-wrap-showcase01' 을 id 로 갖는 그리드로 생성됩니다.

DOM 요소를 확인하면 다음과 같이 그리드 생성 div의 id 를 확인 할 수 있습니다.

```
<div id="aui-pivot-wrap-showcase01" style="position: relative; height: 560px;">
  <div id="aui-pivot-wrap-showcase01__aui_pivot_filter_panel" class="aui-pivot-fi
  <div class="aui-pivot" tabindex="0" style="position: relative; height: 560px; width: 100%;>
```

2. 그리드에 접근하여 메소드 사용하기

React.js 에서 제공하는 useRef Hook 을 이용하여 다음처럼 ref 를 참조하도록 합니다.

```
// 그리드 객체
const myPivot = useRef();
```

위와 같이 useRef 로 myPivot 를 생성한 후 AUIPivot 생성 태그에 ref 로 설정합니다.

```
<AUIPivot ref={myPivot} />
```

그러면 언제든지 리액트 자신의 스코프에서 다음처럼 피벗 그리드에 접근 할 수 있습니다.

```
const pivot = myPivot.current;
pivot.setRowFields(['REGION', 'NAME', 'MODEL']);
```

3. 리액트에서 AUIPivotReact.jsx 사용 예시

```
import { useEffect, useRef } from 'react';
// AUIPivot 컴포넌트
import AUIPivot from '@static/AUIPivot-React/AUIPivotReact';
// 엑셀 다운로드를 브라우저에서 처리하기 위한 file-saver
import 'file-saver';

const TreeLayout = () => {
  // 그리드 객체
  const myPivot = useRef();

  // 피벗 속성 정의
  const pivotProps = {
    width: '100%',
    height: 560
  };

  useEffect(() => {
    console.log('TreeLayout 마운트됨');
    // 초기 이벤트 바인딩
    setupGridEvents();
    const pivot = myPivot.current;

    // 원시 데이터 요청
    fetch(`${import.meta.env.BASE_URL}data/car_sales.json`)
      .then((response) => {
        if (!response.ok) throw new Error('HTTP error ' +
response.status);
        return response.json();
      })
      .then((data) => {
        // 요청 완료 후 피벗 초기화
        initPivotData(data);
        pivot.removeAjaxLoader();
      })
      .catch((error) => {
        alert('데이터 요청 실패: ' + error.message);
      });

    return () => {
      console.log('TreeLayout 언마운트됨');
    };
  }, []);

  // 피벗 그리드 이벤트 세팅
  const setupGridEvents = () => {
    const pivot = myPivot.current;
```

```

// 피벗 그리드 이벤트 바인딩
pivot.bind(['pivotBegin', 'pivotComplete'], (event) => {
  console.log(event.type);
  switch (event.type) {
    case 'pivotBegin':
      console.time('피벗 보고서 시간');
      break;
    case 'pivotComplete':
    default:
      console.timeEnd('피벗 보고서 시간');
  }
});

// 피벗 그리드 해당 데이터로 초기화 및 피벗팅 시작
const initPivotData = (data) => {
  const pivot = myPivot.current;

  // 행 필드
  pivot.setRowFields(['REGION', 'NAME', 'MODEL']);

  // 열 필드
  pivot.setColumnFields(['DATE_QTR', 'DATE_MONTH']);

  // 값 필드
  pivot.setValueFields([
    { dataField: 'TOTAL', operation: 'SUM', formatString: '#,##0' },
    { dataField: 'COUNT', operation: 'SUM', formatString: '#,##0' }
  ]);

  // 필터 필드
  pivot.setFilterFields(['COLOR']);

  // 데이터 중 날짜 필드를 명시하여 연, 반기, 분기 등으로 나눠 표시하도록
  pivot.setDateTypeField('DATE');

  // 주어진 JSON 데이터의 필드명을 구분하기 쉽게 alias 를 지정합니다.
  // 만약 완전히 다른 데이터로 다시 설정된다면 alias 들도 다시 설정하십시오.
  // [{"REGION": "서울 지점", "NAME": "아반테", "MODEL": "1.6
  GDi", "COLOR": "Blue", "PRICE": 1384000, "COUNT": 1, "DATE": "2015/01/01", "TOTAL": 13
  84000}, {
    pivot.setFieldAlias({
      REGION: '판매 지점',
      NAME: '차종',
      MODEL: '모델',
      COLOR: '색상',

```

```

        PRICE: '가격',
        COUNT: '판매 대수',
        TOTAL: '매출액',
        DATE: '일',
        DATE_YEAR: '년',
        DATE_HALF: '반기',
        DATE_QTR: '분기',
        DATE_MONTH: '월'
    });

    // 피봇그리드에 데이터 삽입
    pivot.setGridData(data);
};

const openPivotPanel = () => {
    const pivot = myPivot.current;

    // 이미 피벗 필드 패널이 생성된 경우 보이기로 전환.
    if (pivot.isCreatedPivotPanel()) {
        pivot.showPivotPanel();
    } else {
        const props = {
            // 피벗 패널 사용을 위해 피벗 패널이 생성될 DIV의 아이디를 지정함.
            pivotPanelId: '#pivot_show_tree',
            // 피벗 패널의 상단 필드 리스트 영역의 크기 비율 50%로 지정
            fieldListAreaRatio: 0.5,
            // 리사이징 가능 여부
            resizableFieldPanel: true,
            // 이동 가능 여부
            movableFieldPanel: true,
            // 패널 width, height 지정 안하면.. 기본적으로 #pivot_panel의
            width, height를 따름.
            panelWidth: 400,
            panelHeight: 600,
            // 리사이징 할 때 피벗 필드 패널 최소 사이즈
            minFieldPanelWidth: 300,
            minFieldPanelHeight: 400
        };

        // 실제 피벗 필드 패널 생성
        pivot.createPivotPanel(props);
    }
};

const exportClick = () => {
    const pivot = myPivot.current;
    // 내보내기 속성

```

```

const exportProps = {
  // 진행바 표시
  progressBar: true
};

// 내보내기 실행
pivot.exportToXlsx(exportProps);
};

return (
  <div className="desc">
    <h2>트리형 피벗 출력 방식</h2>
    <div>
      <button onClick={openPivotPanel}>피벗 패널
열기(생성하기)</button>
      <button onClick={exportClick}>엑셀로 내보내기</button>
    </div>

    { /* 피벗 패널이 생성될 곳 위치 DIV 지정 */ }
    <div id="pivot_show_tree"></div>

    { /* AUIPivot 컴포넌트 설정 */ }
    <AUIPivot
      ref={myPivot}
      pivotProps={pivotProps}
    />
  </div>
);
};

export default TreeLayout;

```

- 감사합니다. -